SE233 ADVANCED PROGRAMMING • COMPLETE CODEBASE GUIDE

คู่มือเจาะลึกซอร์สโค้ดฉบับสมบูรณ์: Vector Magic

อธิบายโค้ดทีละไฟล์ ทีละฟังก์ชัน และทีละบรรทัด พร้อมแผนผังการรับ-ส่งข้อมูลเชื่อมโยงทั้งระบบ (ครบทั้ง 20 ไฟล์)

สารบัญโครงสร้างไฟล์ในโปรเจกต์ (20 ไฟล์)

บทนำ: สถาปัตยกรรมและการไหลของข้อมูล (Data Flow)

- แผนผังการทำงานตั้งแต่เปิดโปรแกรมจนถึงบันทึกไฟล์ SVG

บทที่ 1: จุดเริ่มต้นของโปรแกรมและโมดูล (Entry & Module)

1. `module-info.java` — กำหนดสิทธิ์โมดูล JavaFX & AWT
2. `Launcher.java` — คลาสหลักเปิดหน้าต่างโปรแกรม

บทที่ 2: โครงสร้างข้อมูลและโมเดล (Data Models)

3. `DetailLevel.java` — Enum ระดับความละเอียด (Low/Med/High)
4. `ColorMode.java` — Enum โหมดสี (Unlimited/Custom)
5. `ConversionSettings.java` — การตั้งค่าและเงื่อนไข Task 9
6. `ImageItem.java` — โมเดลเก็บข้อมูลรูปภาพและผลลัพธ์ SVG

บทที่ 3: เซอร์วิสไฟล์และการประมวลผลสี/ภาพเบื้องต้น

7. `ZipService.java` — ตรวจสอบและแตกไฟล์รูปจาก ZIP
8. `ImagePreconditioner.java` — กรอง Noise และปรับขอบคมชัด
9. `ColorDetector.java` — นับสี หาสีพื้นหลัง และสกัดสีเด่น
10. `PaletteQuantizer.java` — สร้างพาเลตต์สีและแปลงเป็น Byte

บทที่ 4: เครื่องยนต์แปลงเวกเตอร์หลักและการวาด SVG

11. `VectorizationService.java` — ตัวกลางควบคุมการแปลงเวกเตอร์
12. `ImageTracer.java` — เอนจินคณิตศาสตร์คำนวณเส้นโค้ง
13. `SvgViewRenderer.java` — แปลงโค้ด SVG เป็นรูปทรง JavaFX

บทที่ 5: ระบบ Multi-threading และการส่งออกไฟล์

14. `BatchProcessingService.java` — ประมวลผลกลุ่ม (Task 7 & 9)
15. `FileExportService.java` — บันทึกไฟล์ .svg และเก็บประวัติ

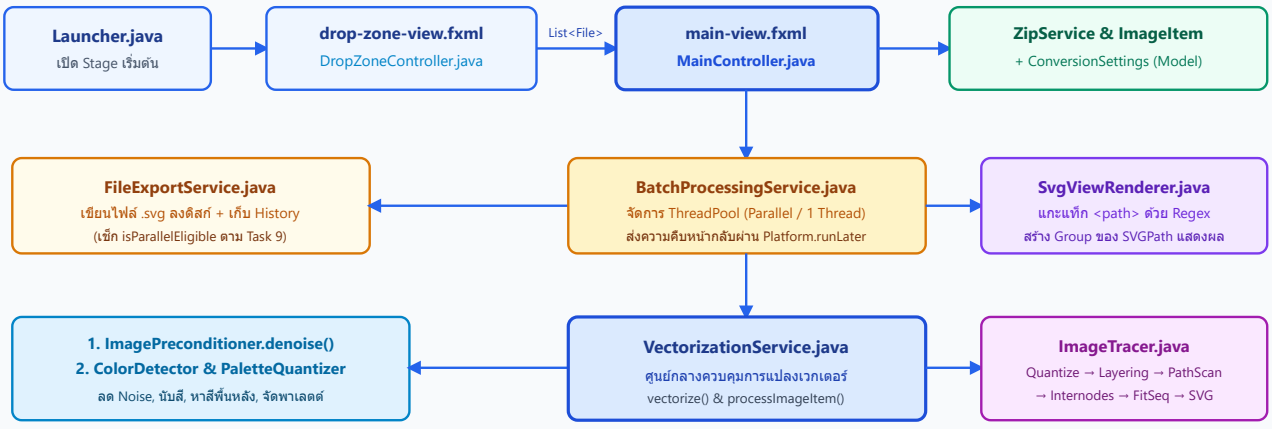
บทที่ 6: ส่วนควบคุมหน้าจอ (Controllers)

16. `DropZoneController.java` — ควบคุมหน้าลากวางไฟล์
17. `MainController.java` — ควบคุมหน้าแก้ไขหลักทั้งหมด

บทที่ 7: ไฟล์ออกแบบหน้าจอและสไตล์ (FXML & CSS)

18. `drop-zone-view.fxml` — หน้าจอรับไฟล์เริ่มต้น
19. `main-view.fxml` — หน้าจอแก้ไขหลัก (Main Editor)
20. `styles.css` — สไตล์ชีตตกแต่งหน้าจอ

แผนผังการเชื่อมโยงและการส่งข้อมูลระหว่างไฟล์ (System Data Flow Diagram)



บทที่ 1: จุดเริ่มต้นของโปรแกรมและโมดูล (Module & Entry Point)

ประกอบด้วยไฟล์ที่ทำหน้าที่กำหนดสิทธิ์โมดูล Java และเปิดหน้าต่างแรกของแอปพลิเคชัน

ไฟล์ที่ 1: module-info.java

`src/main/java/module-info.java`

> **บทบาทหน้าที่:** ประกาศโมดูลของโปรเจกต์ตามระบบ Java Platform Module System (JPMS) เพื่อระบุว่าโปรเจกต์ต้องพึ่งพาไลบรารีใดบ้าง และอนุญาตให้ JavaFX เข้าถึงคลาสภายในโปรเจกต์ได้

```
1: module se233.vertor_magic {
2:     requires javafx.controls;
3:     requires javafx.fxml;
4:     requires java.desktop;
5:
6:     opens se233.vertor_magic to javafx.fxml;
7:     opens se233.vertor_magic.controller to javafx.fxml;
8:     exports se233.vertor_magic;
9:     exports se233.vertor_magic.controller;
10:    exports se233.vertor_magic.model;
11:    exports se233.vertor_magic.service;
12: }
```

- **บรรทัด 1:** ตั้งชื่อโมดูลว่า `se233.vertor_magic`
- **บรรทัด 2–3:** `requires javafx.controls;` และ `requires javafx.fxml;` ดึงไลบรารี UI ของ JavaFX (เช่น `Button`, `Label`, `ScrollPane`) และตัวอ่านไฟล์ FXML (`FXMLLoader`) เข้ามาใช้งาน
- **บรรทัด 4:** `requires java.desktop;` ดึงไลบรารีจัดการรูปภาพระดับพิกเซล (`BufferedImage`, `ImageIO`, `Color`) และระบบเปิดโฟลเดอร์ของ OS (`Desktop`)
- **บรรทัด 6–7:** `opens ... to javafx.fxml;` เปิดสิทธิ์ให้ `FXMLLoader` ใช้ Reflection เข้าถึงตัวแปร `private` ที่ติดป้าย `@FXML` ในแพ็กเกจหลักและแพ็กเกจ `controller` ได้
- **บรรทัด 8–11:** `exports ...` เปิดให้คลาสในแพ็กเกจทั้ง 4 สามารถเรียกใช้งานข้ามกันและถูกเรียกจากชุดทดสอบ (JUnit Tests) ได้

ไฟล์ที่ 2: Launcher.java

`src/main/java/se233/vertor_magic/Launcher.java`

> **บทบาทหน้าที่:** เป็นจุดเริ่มต้นหลัก (Main Entry Point) ของโปรแกรม สืบทอดคลาส `Application` ของ JavaFX ทำหน้าที่สร้างหน้าต่างหลัก (`Stage`) และโหลดหน้าจอแรก (`drop-zone-view.fxml`)

รับข้อมูลจาก: JVM ตอนสั่งรันโปรแกรม | **ส่งต่อไปยัง: drop-zone-view.fxml & DropZoneController**

```
1: package se233.vertor_magic;
2:
3: import javafx.application.Application;
4: import javafx.fxml.FXMLLoader;
5: import javafx.scene.Scene;
6: import javafx.stage.Stage;
```

```
 7: import java.io.IOException;
 8:
10: public class Launcher extends Application {
12:     @Override
13:     public void start(Stage stage) throws IOException {
15:         FXMLLoader fxmlLoader = new FXMLLoader(Launcher.class.getResource("drop-zone-
view.fxml"));
16:         Scene scene = new Scene(fxmlLoader.load(), 1180, 780);
17:         stage.setTitle("Vector Magic - Raster to Vector Converter (SE233)");
18:         stage.setMinWidth(900);
19:         stage.setMinHeight(600);
20:         stage.setScene(scene);
21:         stage.show();
22:     }
24: }
```

- **บรรทัด 10:** `extends Application` ทำให้คลาสนี้มีวงจรชีวิตเป็นแอปพลิเคชัน JavaFX
- **บรรทัด 13:** เมธอด `start(Stage stage)` ถูกเรียกโดย JavaFX Application Thread พร้อมส่งกรอบหน้าต่างหลัก `stage` เข้ามา
- **บรรทัด 15:** ค้นหาไฟล์ `drop-zone-view.fxml` จากโฟลเดอร์ `resources/se233/vertor_magic/` แล้วสร้างตัวโหลด `FXMLLoader`
- **บรรทัด 16:** สั่ง `fxmlLoader.load()` เพื่ออ่าน XML สร้างโหนด UI ทั้งหมด และผูกเข้ากับ `DropZoneController` อัตโนมัติ จากนั้นนำมาสร้างเป็น `Scene` ขนาดเริ่มต้น `1180 x 780` พิกเซล
- **บรรทัด 17–21:** ตั้งชื่อหน้าต่าง กำหนดขนาดหน้าต่างขั้นต่ำ (`900 x 600`) นำ `scene` ใส่ลงใน `stage` แล้วสั่ง `stage.show()` เพื่อแสดงผล

บทที่ 2: โครงสร้างข้อมูลและโมเดล (Data Models)

คลาสในแพ็กเกจ se233.vertor_magic.model สำหรับเก็บสถานะของรูปภาพและการตั้งค่าการแปลงเวกเตอร์

ไฟล์ที่ 3: DetailLevel.java

`src/main/java/se233/vertor_magic/model/DetailLevel.java`

> **บทบาทหน้าที่:** เป็น `enum` กำหนดระดับความละเอียดของการสร้างเส้นเวกเตอร์ 3 ระดับตามโจทย์ **Task 4** ได้แก่ `LOW` , `MEDIUM` , และ `HIGH`

```
 9: public enum DetailLevel {
10:     LOW("Low"),
11:     MEDIUM("Medium"),
12:     HIGH("High");
13:
14:     private final String displayName;
16:     DetailLevel(String displayName) { this.displayName = displayName; }
20:     public String getDisplayName() { return displayName; }
24:     @Override public String toString() { return displayName; }
28: }
```

- **บรรทัด 10–12:** นิยามค่าคงที่ 3 ระดับ โดย `LOW` เน้นเส้นเรียบง่ายขนาดไฟล์เล็ก, `MEDIUM` สมดุล (ค่าเริ่มต้น), และ `HIGH` เก็บรายละเอียดคมกริบ
- **บรรทัด 14–27:** เก็บชื่อสำหรับแสดงผล (`displayName`) และ Override `toString()` เพื่อให้นำไปแสดงข้อความบน Label ของ UI ได้โดยตรง

ไฟล์ที่ 4: ColorMode.java

`src/main/java/se233/vertor_magic/model/ColorMode.java`

> **บทบาทหน้าที่:** เป็น `enum` กำหนดโหมดการใช้สีตามโจทย์ **Task 5** ได้แก่ `UNLIMITED` (ไม่จำกัดสี) และ `CUSTOM` (จำกัดสี 2–5 สี)

```
 8: public enum ColorMode {
 9:     UNLIMITED("Unlimited"),
10:     CUSTOM("Custom");
12:     private final String displayName;
14:     ColorMode(String displayName) { this.displayName = displayName; }
18:     public String getDisplayName() { return displayName; }
22:     @Override public String toString() { return displayName; }
26: }
```

ไฟล์ที่ 5: ConversionSettings.java

`src/main/java/se233/vertor_magic/model/ConversionSettings.java`

> **บทบาทหน้าที่:** เก็บพารามิเตอร์การตั้งค่าการแปลงเวกเตอร์ทั้งหมดไว้ในอ็อบเจกต์เดียว และมีเมธอด `isParallelEligible()` สำหรับตัดสินใจว่าจะรันแบบมัลติเธรดหรือเธรดเดี่ยวตามโจทย์ **Task 9**

รับค่าจาก: **MainController (เมื่อผู้ใช้คลิกเปลี่ยนปุ่ม)**

ส่งต่อไปยัง: **BatchProcessingService, FileExportService, VectorizationService**

```
 8: public class ConversionSettings {
 9:     private DetailLevel detailLevel;
10:     private ColorMode colorMode;
11:     private int customColorCount;
12:     private boolean removeBackground;
13:
14:     public ConversionSettings() {
17:         this.detailLevel = DetailLevel.MEDIUM;
18:         this.colorMode = ColorMode.CUSTOM;
19:         this.customColorCount = 2;
20:         this.removeBackground = false;
21:     }
23:     public ConversionSettings(DetailLevel detailLevel, ColorMode colorMode, int customColorCount,
boolean removeBackground) {
24:         this.detailLevel = detailLevel != null ? detailLevel : DetailLevel.MEDIUM;
25:         this.colorMode = colorMode != null ? colorMode : ColorMode.CUSTOM;
26:         this.customColorCount = Math.max(2, Math.min(5, customColorCount));
27:         this.removeBackground = removeBackground;
28:     }
30:     public ConversionSettings(ConversionSettings other) { ... }
49:     public boolean isParallelEligible() {
50:         return detailLevel == DetailLevel.MEDIUM
51:                 && colorMode == ColorMode.CUSTOM
52:                 && customColorCount >= 2
53:                 && customColorCount <= 5;
54:     }
```

- **บรรทัด 14–21 (Default Constructor):** กำหนดค่าเริ่มต้นตามโจทย์ **Task 7** คือ `DetailLevel.MEDIUM`, `ColorMode.CUSTOM`, `customColorCount = 2`, และ `removeBackground = false`
- **บรรทัด 26 & 77:** ใช้ `Math.max(2, Math.min(5, customColorCount))` เพื่อป้องกันไม่ให้จำนวนสีหลุดออกนอกช่วง 2 ถึง 5 สี
- **บรรทัด 30–42 (Copy Constructor):** สร้างสำเนาอ็อบเจกต์ใหม่ เพื่อไม่ให้รูปภาพหลายรูปแชร์ Reference การตั้งค่าตัวเดียวกันโดยไม่ตั้งใจ
- **บรรทัด 49–54 (`isParallelEligible`):** เมธอดหัวใจของ **Task 9** คืนค่า `true` เมื่อ `detailLevel` เป็น `MEDIUM` และ `colorMode` เป็น `CUSTOM` (2–5 สี) เท่านั้น หากเป็นค่าอื่นจะคืนค่า `false` ทันที
- **บรรทัด 88–102 (`equals` & `hashCode`):** เปรียบเทียบความเท่ากันของค่าทั้ง 4 ตัว เพื่อให้ระบบส่งออกไฟล์เช็กได้ว่าต้องคำนวณ SVG ใหม่หรือไม่

ไฟล์ที่ 6: ImageItem.java

`src/main/java/se233/vertor_magic/model/ImageItem.java`

> **บทบาทหน้าที่:** เป็นอ็อบเจกต์ตัวแทนของรูปภาพแต่ละไฟล์ที่ผู้ใช้นำเข้า เก็บทั้งภาพต้นฉบับ พิกเซลสำหรับคำนวณ ข้อมูลสี การตั้งค่าประจำภาพ และข้อความโค้ด SVG ที่แปลงเสร็จแล้ว

```
17: public class ImageItem {
18:     private final File sourceFile;
19:     private final String originalFileName;
```

```
20:     private BufferedImage bufferedImage;
21:     private Image fxImage;
22:     private String svgContent;
23:     private int detectedColorCount;
24:     private List<Color> detectedPalette = new ArrayList<>();
25:     private ConversionSettings settings;
26:     private boolean isProcessed;
27:     private String processingError;
28:
29:     public ImageItem(File file) throws IOException {
30:         this.sourceFile = file;
31:         this.originalFileName = file.getName();
32:         this.bufferedImage = ImageIO.read(file);
33:         if (this.bufferedImage == null) {
34:             throw new IOException("Unsupported or corrupt image file: " + file.getName());
35:         }
36:         this.fxImage = new Image(file.toURI().toString());
37:         this.settings = new ConversionSettings();
38:         this.isProcessed = false;
39:     }
60:     public String getBaseName() {
61:         int dotIdx = originalFileName.lastIndexOf('.');
62:         return dotIdx > 0 ? originalFileName.substring(0, dotIdx) : originalFileName;
63:     }
92:     public void setSvgContent(String svgContent) {
93:         this.svgContent = svgContent;
94:         this.isProcessed = svgContent != null && !svgContent.isEmpty();
95:     }
```

- **บรรทัด 29–39:** โหลดไฟล์รูปภาพขึ้นหน่วยความจำ 2 รูปแบบ คือ `BufferedImage` (ส่งให้ `ColorDetector` และ `VectorizationService` คำนวณพิกเซล) และ `fxImage` (ส่งให้ `MainController` แสดงผลในช่อง Original Image)
- **บรรทัด 60–63 (`getBaseName`):** ตัดนามสกุลไฟล์เดิมออก (เช่น `"sample.png"` เหลือ `"sample"`) เพื่อนำไปตั้งชื่อไฟล์ `.svg` ตอนส่งออก
- **บรรทัด 92–95 (`setSvgContent`):** บันทึกโค้ด SVG ที่สร้างเสร็จแล้ว พร้อมเปลี่ยนสถานะ `isProcessed` เป็น `true`

บทที่ 3: เซอร์วิสจัดการไฟล์และการประมวลผลสี/ภาพเบื้องต้น

ประกอบด้วย ZipService, ImagePreconditioner, ColorDetector และ PaletteQuantizer

ไฟล์ที่ 7: ZipService.java

`src/main/java/se233/vertor_magic/service/ZipService.java`

> **บทบาทหน้าที่:** รับผิดชอบ **Task 1** ในการตรวจสอบนามสกุลไฟล์ และแตกไฟล์รูปภาพ (`.jpg` , `.jpeg` , `.png`) ออกจากไฟล์ `.zip` ลงในโฟลเดอร์ชั่วคราวอย่างปลอดภัย

รับข้อมูลจาก: MainController.importFiles(rawFiles)

คืนค่า: List<File> ของรูปภาพที่แตกออกมา

```
 17:     private static final String[] SUPPORTED_EXTENSIONS = {".jpg", ".jpeg", ".png"};
 26:     public static List<File> extractImages(File zipFile) throws IOException {
 31:         List<File> extractedFiles = new ArrayList<>();
 33:         Path tempDir = Files.createTempDirectory("vector_magic_" +
UUID.randomUUID().toString().substring(0, 8));
 36:         try (ZipInputStream zis = new ZipInputStream(new FileInputStream(zipFile))) {
 37:             ZipEntry entry;
 40:             while ((entry = zis.getNextEntry()) != null) {
 41:                 if (entry.isDirectory()) { zis.closeEntry(); continue; }
 46:                 String entryName = entry.getName();
 49:                 if (entryName.startsWith("__MACOSX") || entryName.contains("/.") ||
entryName.startsWith(".")) {
 53:                     zis.closeEntry(); continue;
 55:                 }
 58:                 if (isImageExtension(entryName)) {
 60:                     String simpleName = new File(entryName).getName();
 61:                     Path targetFile = tempDir.resolve(simpleName);
 64:                     if (!targetFile.normalize().startsWith(tempDir.normalize())) {
 65:                         throw new IOException("Illegal ZIP entry path: " + entryName);
 66:                     }
 69:                     int counter = 1;
 70:                     while (Files.exists(targetFile)) { ... } // เติม _1, _2 หากชื่อซ้ำ
 84:                     try (OutputStream os = new FileOutputStream(targetFile.toFile())) {
 85:                         byte[] buffer = new byte[8192];
 87:                         while ((len = zis.read(buffer)) > 0) { os.write(buffer, 0, len); }
 90:                     }
 92:                     extractedFiles.add(targetFile.toFile());
 93:                 }
 94:                 zis.closeEntry();
 95:             }
 96:         }
 98:         if (extractedFiles.isEmpty()) {
 99:             throw new IOException("No supported JPG or PNG images found inside ZIP archive: " +
zipFile.getName());
100:         }
102:         return extractedFiles;
103:     }
```

- **บรรทัด 33:** สร้างโฟลเดอร์ชั่วคราวด้วย `Files.createTempDirectory` เพื่อเก็บไฟล์รูปที่แตกออกมา
- **บรรทัด 49–55:** กรองไฟล์ขยะของ macOS (`__MACOSX`) และไฟล์ซ่อนที่ขึ้นต้นด้วยจุดทิ้งไป
- **บรรทัด 64–66:** ตรวจสอบความปลอดภัยว่าพาธปลายทางไม่ออกนอก `tempDir` (ป้องกันช่องโหว่ Zip-Slip)

- **บรรทัด 69–92:** แก้ปัญหาไฟล์ชื่อซ้ำด้วยการเติม `_1`, `_2` แล้วเขียนข้อมูลไบต์ลงดิสก์ทีละ 8 KB (`byte[8192]`)
- **บรรทัด 106–133:** เมธอด `isZipFile(File)`, `isImageFile(File)` และ `isImageExtension(String)` ใช้ตรวจสอบนามสกุลไฟล์

ไฟล์ที่ 8: ImagePreconditioner.java

`src/main/java/se233/vertor_magic/service/ImagePreconditioner.java`

> **บทบาทหน้าที่:** ปรับปรุงคุณภาพของภาพพิกเซลก่อนนำไปแปลงเป็นเวกเตอร์ โดยลบจุดรบกวน (Noise) ในบริเวณพื้นเรียบ และดึงขอบที่ตัดกันให้คมชัดขึ้นด้วย 3x3 Shock Filter

ถูกเรียกโดย: VectorizationService.vectorize() **คืนค่า: BufferedImage ที่ผ่านการกรองแล้ว**

```
 21:     public static BufferedImage denoise(BufferedImage src, int colorTolerance) {
 26:         int[] srcPix = src.getRGB(0, 0, w, h, null, 0, w);
 27:         int[] dstPix = new int[w * h];
 29:         for (int y = 0; y < h; y++) {
 30:             for (int x = 0; x < w; x++) {
 39:                 if (cr >= 250 && cg >= 250 && cb >= 250) {
 40:                     dstPix[idx] = (ca << 24) | 0x00FFFFFF;
 41:                     continue;
 42:                 }
 50:                 int cLum = (cr * 77 + cg * 150 + cb * 29) >> 8;
 56:                 for (int dy = -1; dy <= 1; dy++) {
 59:                     for (int dx = -1; dx <= 1; dx++) { ... } // สำรวจเพื่อนบ้าน 3x3
 84:                 }
 86:                 int maxSpan = Math.max(maxR - minR, Math.max(maxG - minG, maxB - minB));
 89:                 if (maxSpan >= 36) {
 90:                     int meanLum = sumLum / count;
 92:                     int snappedRgb = (cLum <= meanLum) ? minRgb : maxRgb;
 93:                     dstPix[idx] = (ca << 24) | snappedRgb;
 94:                 } else if (maxSpan <= colorTolerance) {
 96:                     dstPix[idx] = (ca << 24) | ((sumR / count) << 16) | ((sumG / count) << 8) |
(sumB / count);
 97:                 } else {
 99:                     dstPix[idx] = center;
100:                 }
101:             }
102:         }
106:         return result;
107:     }
```

- **บรรทัด 39–42:** ปรับพิกเซลสีเกือบขาว (R, G, B ≥ 250) ให้เป็นสีขาวบริสุทธิ์ เพื่อไม่ให้พื้นหลังเกิดคราบสีเพี้ยน
- **บรรทัด 50–86:** คำนวณความสว่าง (Luminance) และสำรวจพิกเซลรอบข้างขนาด 3x3 เพื่อหาความต่างสีสูงสุด (`maxSpan`)
- **บรรทัด 89–100:**
  - ถ้า `maxSpan >= 36` (บริเวณขอบตัดกัน): ดึงสีเข้าหาพิกเซลเพื่อนบ้านที่มืดสุดหรือสว่างสุด (`minRgb` / `maxRgb`) ทำให้ขอบคมกริบ
  - ถ้า `maxSpan <= colorTolerance` (บริเวณสีเรียบ): เฉลี่ยค่าสีในหน้าต่าง 3x3 เพื่อลบ Noise ของ JPEG
  - ถ้าอยู่ระหว่างกลาง (บริเวณไล่เฉดสี): คงค่าสีเดิมไว้

ไฟล์ที่ 9: ColorDetector.java

`src/main/java/se233/vertor_magic/service/ColorDetector.java`

> **บทบาทหน้าที่:** วิเคราะห์ข้อมูลสีของรูปภาพตามโจทย์ **Task 5** และ **Task 6** มี 4 เมธอดสำคัญสำหรับนับจำนวนสี คำนวณเพดานสี Custom ตรวจหาสีพื้นหลัง และสกัดสีเด่น

1. countUniqueColors(BufferedImage image) — บรรทัด 16–40

```
24:         Set<Integer> uniqueColors = new HashSet<>();
26:         int step = Math.max(1, (int) Math.sqrt((width * height) / 50000.0));
28:         for (int y = 0; y < height; y += step) {
29:             for (int x = 0; x < width; x += step) {
32:                 uniqueColors.add(image.getRGB(x, y) & 0x00FFFFFF);
33:                 if (uniqueColors.size() > 500) return uniqueColors.size();
37:             }
38:         }
```

- เก็บค่าสี RGB ที่ไม่ซ้ำกันลงใน `HashSet` หากเกิน 500 สีจะหยุดนับทันทีเพื่อให้ทำงานได้รวดเร็วแม้รูปมีขนาดใหญ่

2. getCustomColorUpperLimit(int detectedColorCount) — บรรทัด 46–51

```
46:     public static int getCustomColorUpperLimit(int detectedColorCount) {
47:         if (detectedColorCount >= 5) return 5;
50:         return Math.max(2, detectedColorCount);
51:     }
```

- กำหนดเพดานสูงสุดของ Spinner เลือกสีในโหมด Custom: ถ้ารูปมีมากกว่า 5 สี จะล็อกเพดานสูงสุดไว้ที่ `5` แต่ถ้ารูปมีน้อยกว่า 5 สี จะล็อกตามจำนวนสีจริงของรูป (ต่ำสุดคือ 2)

3. detectBackgroundColor(BufferedImage image) — บรรทัด 56–94

```
56:     public static Color detectBackgroundColor(BufferedImage image) {
61:         Map<Integer, Integer> borderFreq = new HashMap<>();
64:         int[] sampleX = {0, width - 1, width / 2};
65:         int[] sampleY = {0, height - 1, height / 2};
77:         for (int x = 0; x < width; x += Math.max(1, width / 10)) { ... }
93:         return new Color(dominantRgb, true);
94:     }
```

- สำรวจพิกเซลบริเวณมุมทั้ง 4 และขอบบน-ล่างของรูปภาพ เพื่อหาสีที่มีความถี่สูงสุดเป็นสีพื้นหลัง แล้วส่งให้ `PaletteQuantizer` และ `VectorizationService` ใช้ระบุเลเยอร์พื้นหลัง

4. extractDominantColors(BufferedImage image, int maxColors) — บรรทัด 100–174

```
120:         int qr = (r >> 3) << 3; int qg = (g >> 3) << 3; int qb = (b >> 3) << 3;
148:         int[] distanceThresholds = {80, 50, 30, 15, 0};
157:         double dist = Math.sqrt(dr * dr + dg * dg + db * db);
```

- จัดกลุ่มสีแบบ 5-bit เรียงลำดับจากสีที่พบบ่อยที่สุด แล้วคัดเลือกสีที่มีระยะห่างทางเรขาคณิต (Euclidean Distance) แตกต่างกันชัดเจนตามจำนวน `maxColors` (2–5 สีสำหรับโหมด Custom)

ไฟล์ที่ 10: PaletteQuantizer.java

`src/main/java/se233/vertor_magic/service/PaletteQuantizer.java`

> **บทบาทหน้าที่:** สกัดพาเลตต์สีแบบละเอียดสำหรับโหมด `UNLIMITED` (10, 20, หรือ 40 สีตาม DetailLevel) และแปลงลิสต์ของ `Color` ให้เป็นอาร์เรย์ `byte[][]` สำหรับส่งเข้า `ImageTracer`

```
 19:     public static List<Color> quantize(BufferedImage image, int targetColors) {
 55:         List<Color> result = new ArrayList<>();
 57:         Color bgColor = ColorDetector.detectBackgroundColor(image);
 58:         result.add(new Color(bgColor.getRed(), bgColor.getGreen(), bgColor.getBlue()));
 61:         double minDistance = Math.max(14.0, 160.0 / Math.pow(targetColors, 0.45));
 64:         addCandidatesByDistance(sorted, result, targetColors, minDistance);       // Pass 1: สีหลัก
 69:         addCandidatesByDistance(sorted, result, targetColors, relaxedDist);       // Pass 2: สีรอง
 74:         addCandidatesByDistance(sorted, result, targetColors, 1.0);               // Pass 3: เฉดไล่ระดับ
 77:         return result;
 78:     }
111:     public static byte[][] toTracerPalette(List<Color> colors) {
116:         palette[i][0] = (byte) (c.getRed() - 128);
117:         palette[i][1] = (byte) (c.getGreen() - 128);
118:         palette[i][2] = (byte) (c.getBlue() - 128);
119:         palette[i][3] = (byte) (c.getAlpha() - 128);
121:         return palette;
122:     }
```

- **บรรทัด 57–58:** เรียก `ColorDetector.detectBackgroundColor(image)` เพื่อล็อกสีพื้นหลังของภาพเป็นสีแรกในพาเลตต์เสมอ
- **บรรทัด 61–75:** คัดเลือกสีลงพาเลตต์ 3 รอบตามระยะห่างของสี เพื่อให้ได้ทั้งสีหลักที่ตัดกันและสีไล่ระดับที่นุ่มนวล
- **บรรทัด 111–122 (`toTracerPalette`):** แปลงค่าสี RGB ช่วง `0..255` ให้เป็น `byte` ช่วง `-128..127` ด้วยการลบ `128` ตามรูปแบบข้อมูลของ `ImageTracer`

บทที่ 4: เครื่องยนต์แปลงเวกเตอร์หลักและการวาด SVG

ประกอบด้วย VectorizationService, ImageTracer และ SvgViewRenderer

ไฟล์ที่ 11: VectorizationService.java

`src/main/java/se233/vertor_magic/service/VectorizationService.java`

> **บทบาทหน้าที่:** เป็นศูนย์กลางประสานงานการแปลงภาพ (Orchestrator) เชื่อมต่อระหว่าง `ImagePreconditioner`, `ColorDetector`, `PaletteQuantizer` และ `ImageTracer`

```
17:     public static String vectorize(BufferedImage image, ConversionSettings settings) throws
Exception {
26:         int tolerance = getDenoiseTolerance(settings.getDetailLevel());
27:         BufferedImage conditionedImage = ImagePreconditioner.denoise(image, tolerance);
29:         HashMap<String, Float> options = ImageTracer.checkoptions(null);
35:         applyDetailLevelSettings(options, settings.getDetailLevel());
37:         byte[][] palette = preparePalette(conditionedImage, options, settings);
39:         ImageTracer.ImageData imgd = ImageTracer.loadImageData(conditionedImage);
40:         ImageTracer.IndexedImage ii = ImageTracer.imagedataToTracedata(imgd, options, palette);
42:         Color detectedBgColor = ColorDetector.detectBackgroundColor(conditionedImage);
43:         int bgLayerIdx = detectBackgroundLayerIndex(ii, detectedBgColor);
45:         return ImageTracer.getsvgstring(ii, options, settings.isRemoveBackground(), bgLayerIdx);
46:     }
```

- **บรรทัด 26–27 (`getDenoiseTolerance` & `denoise`):** กำหนดความแรงในการลด Noise ตามระดับความละเอียด (`HIGH=8`, `MEDIUM=14`, `LOW=20`) แล้วส่งเข้า `ImagePreconditioner.denoise()`
- **บรรทัด 35 (`applyDetailLevelSettings` — บรรทัด 76–105):** ตั้งค่าความคลาดเคลื่อนของเส้นตรง (`ltres`), เส้นโค้ง (`qtres`), และขนาดจุดขั้นต่ำ (`pathomit`):
  - `LOW` : `ltres=2.0f, qtres=1.6f, pathomit=8.0f`
  - `MEDIUM` : `ltres=1.2f, qtres=1.0f, pathomit=6.0f`
  - `HIGH` : `ltres=0.8f, qtres=0.6f, pathomit=4.0f`
- **บรรทัด 37 (`preparePalette` — บรรทัด 107–147):**
  - ถ้าเป็น `CUSTOM` : สกัดสีเด่น 2–5 สีจาก `ColorDetector.extractDominantColors()`
  - ถ้าเป็น `UNLIMITED` : สกัดสีด้วย `PaletteQuantizer.quantize()` (Low=10, Medium=20, High=40 สี)
- **บรรทัด 42–45 (`detectBackgroundLayerIndex` — บรรทัด 149–213):** นับความถี่ของเลเยอร์ที่ขอบภาพทั้ง 4 ด้าน และให้คะแนนพิเศษ (+10.0) แก่เลเยอร์ที่มีสีตรงกับ `detectedBgColor` เพื่อส่งหมายเลขเลเยอร์พื้นหลังไปให้ `ImageTracer.getsvgstring()` ตัดออกหากผู้ใช้ติ๊ก Remove Background
- **บรรทัด 48–61 (`processImageItem`):** เมธอดสะพานเชื่อมที่รับ `ImageItem` เข้ามา นับสีของภาพ เรียก `vectorize()` แล้วนำโค้ด SVG ที่ได้เก็บกลับเข้า `item.setSvgContent(svg)`

ไฟล์ที่ 12: ImageTracer.java

`src/main/java/se233/vertor_magic/ImageTracer.java`

> **บทบาทหน้าที่:** เอนจินคณิตศาสตร์สำหรับแปลงข้อมูลพิกเซล (`ImageData`) ให้กลายเป็นสมการเส้นเวกเตอร์และประกอบเป็นโค้ด XML มาตรฐานของ SVG

```
213:     public static IndexedImage imagedataToTracedata (ImageData imgd, HashMap<String,Float>
options, byte [][] palette){
215:         IndexedImage ii = colorquantization(imgd, palette, options);
217:         int[][][] rawlayers = layering(ii);
219:         ArrayList<ArrayList<ArrayList<Integer[]>>> bps = batchpathscan(rawlayers,(int)
(Math.floor(options.get("pathomit"))));
221:         ArrayList<ArrayList<ArrayList<Double[]>>> bis = batchinternodes(bps);
223:         ii.layers = batchtracelayers(bis,options.get("ltres"),options.get("qtres"));
224:         return ii;
225:     }
```

- **ขั้นที่ 1 — `colorquantization` (บรรทัด 265–364):** จัดกลุ่มสีของทุกพิกเซลเข้ากับสีในพาเลตต์ที่ใกล้เคียงที่สุด และปรับจุดศูนย์กลางสี (K-Means Clustering) ตามจำนวนรอบ `colorquantcycles`
- **ขั้นที่ 2 — `layering` (บรรทัด 436–468):** แยกแต่ละสีออกเป็นตาราง 2 มิติ (Layer) เพื่อเตรียมหาเส้นขอบของแต่ละสี
- **ขั้นที่ 3 — `pathscan` & `batchpathscan` (บรรทัด 505–566):** เดินไต่ตามขอบพิกเซลของแต่ละสีโดยใช้ตาราง Lookup ทิศทาง (Marching Squares) เพื่อสร้างเส้นรอบรูปปิด และละทิ้งเส้นที่มีจำนวนจุดน้อยกว่า `pathomit`
- **ขั้นที่ 4 — `internodes` (บรรทัด 569–656):** คำนวณจุดกึ่งกลางระหว่างพิกเซล และตรวจจับมุมแหลม (Corner Detection) เพื่อไม่ให้มุมของโลโก้หรือตัวอักษรถูกปัดจนมนเสียรูป
- **ขั้นที่ 5 — `tracepath` & `fitseq` (บรรทัด 674–835):** แทนที่ชุดของจุดด้วยเส้นตรง (`L x y`) หากความคลาดเคลื่อนไม่เกิน `ltres` หรือแทนที่ด้วยเส้นโค้งเบซิเยร์กำลังสอง (`Q cx cy x y`) หากความคลาดเคลื่อนไม่เกิน `qtres` หากเกินจะแบ่งครึ่งช่วงแล้วคำนวณซ้ำแบบ Recursive
- **ขั้นที่ 6 — `getsvgstring` & `svgpathstring` (บรรทัด 869–988):** เรียงลำดับเลเยอร์จากพื้นที่ใหญ่ไปเล็ก ข้ามเลเยอร์พื้นหลัง (`bgLayerIdx`) เมื่อ `removeBackground == true` และสร้างสตริง `<svg ...><path ... /></svg>`

ไฟล์ที่ 13: SvgViewRenderer.java

`src/main/java/se233/vertor_magic/service/SvgViewRenderer.java`

> **บทบาทหน้าที่:** แปลงข้อความโค้ด SVG (`String`) ให้เป็นอ็อบเจกต์ `javafx.scene.Group` ที่บรรจุโหนด `SVGPath` เพื่อให้แสดงผลและซูมบนหน้าจอ JavaFX ได้โดยตรง

```
31:     public static Group renderSvgToGroup(String svgContent) {
32:         Group group = new Group();
37:         Matcher pathMatcher = PATH_PATTERN.matcher(svgContent);
38:         while (pathMatcher.find()) {
43:             String d = dMatcher.group(1);
45:             SVGPath svgPath = new SVGPath();
46:             svgPath.setContent(d);
53:             svgPath.setFill(fillColor);
88:             group.getChildren().add(svgPath);
89:         }
91:         return group;
92:     }
```

- **บรรทัด 17–23:** ใช้ Regular Expression จับคู่แท็ก `<path .../>` และดึงค่าแอตทริบิวต์ `d`, `fill`, `stroke`, `stroke-width`, และ `opacity`
- **บรรทัด 38–89:** สร้างอ็อบเจกต์ `SVGPath` ของ JavaFX ทีละเส้น กำหนดสีพื้นและสีเส้นขอบด้วยเมธอด `parseColor()` (รองรับทั้งรูปแบบ `rgb(r,g,b)` และรหัสสีเว็บ) แล้วรวมเข้าไว้ใน `Group` ส่งกลับให้ `MainController`

บทที่ 5: เซอร์วิสประมวลผลแบบกลุ่ม (Multi-threading) และการส่งออกไฟล์

ประกอบด้วย BatchProcessingService และ FileExportService (Task 7, Task 8, Task 9, Task 12)

ไฟล์ที่ 14: BatchProcessingService.java

`src/main/java/se233/vertor_magic/service/BatchProcessingService.java`

> **บทบาทหน้าที่:** จัดการการแปลงภาพล่วงหน้าแบบกลุ่ม (**Task 7: Batch Preprocessing**) และควบคุมการใช้เธรดตามเงื่อนไข (**Task 9: Conditional Parallel Processing**)

```
 24:     public void preprocessBatch(List<ImageItem> items, Consumer<Double> progressCallback,
 26:                                 Consumer<String> statusCallback, Consumer<ImageItem>
itemCompletedCallback,
 28:                                 Runnable onFinishedCallback) {
 29:         processBatch(items, new ConversionSettings(), progressCallback, statusCallback,
itemCompletedCallback, onFinishedCallback);
 30:     }
 35:     public void processBatch(List<ImageItem> items, ConversionSettings settings, ...) {
 51:         boolean qualifyParallel = effectiveSettings.isParallelEligible() && items.size() > 1;
 53:         int threadCount = qualifyParallel ? Math.min(items.size(), Math.max(2,
Runtime.getRuntime().availableProcessors())) : 1;
 58:         currentExecutor = Executors.newFixedThreadPool(threadCount, r -> {
 59:             Thread t = new Thread(r, "Batch-Processor-Thread");
 60:             t.setDaemon(true);
 61:             return t;
 62:         });
 72:         new Thread(() -> {
 75:             for (ImageItem item : items) {
 76:                 Future<?> future = currentExecutor.submit(() -> {
 81:                     VectorizationService.processImageItem(item);
 86:                     int finished = completedCount.incrementAndGet();
 87:                     double progress = (double) finished / total;
 89:                     Platform.runLater(() -> {
 90:                         if (progressCallback != null) progressCallback.accept(progress);
 95:                         if (itemCompletedCallback != null) itemCompletedCallback.accept(item);
 97:                     });
 98:                 });
 99:                 futures.add(future);
100:             }
103:             for (Future<?> f : futures) { f.get(); }
110:             shutdownExecutor();
112:             Platform.runLater(() -> { ... onFinishedCallback.run(); });
118:         }, "Batch-Dispatcher-Thread").start();
119:     }
```

- **บรรทัด 24–30 (`preprocessBatch`):** เรียก `processBatch` ด้วยการตั้งค่าเริ่มต้น (Medium + Custom 2 สี) ทันทีที่ผู้ใช้นำรูปเข้า
- **บรรทัด 51–53:** เช็กเงื่อนไข **Task 9** ด้วย `effectiveSettings.isParallelEligible() && items.size() > 1` หากเป็นจริงจะตั้งจำนวนเธรดตามจำนวนคอร์ของ CPU (Parallel) แต่หากเป็นเท็จจะใช้เพียง `1` เธรด (Sequential)
- **บรรทัด 72–100:** ส่งงานแปลงภาพแต่ละรูปเข้า `currentExecutor` เมื่อแต่ละรูปเสร็จจะเพิ่มตัวนับ `AtomicInteger completedCount` และส่งค่าความคืบหน้ากลับไปอัปเดต UI ผ่าน `Platform.runLater()` เพื่อความปลอดภัยของ

JavaFX Thread

ไฟล์ที่ 15: FileExportService.java `src/main/java/se233/vertor_magic/service/FileExportService.java`

> **บทบาทหน้าที่:** ส่งออกรูปภาพทั้งหมดเป็นไฟล์ `.svg` ลงในโฟลเดอร์ที่ผู้ใช้เลือก (**Task 8**), ใช้เงื่อนไขมัลติเธรดตาม **Task 9**, และบันทึกประวัติการส่งออก (**Task 12**)

```
 27:     public static class ExportRecord {
 28:         public final String timestamp;
 29:         public final File outputDirectory;
 30:         public final List<File> exportedFiles;
 39:     }
 52:     public void exportAll(List<ImageItem> items, File outputDirectory, ConversionSettings
activeSettings, ...) {
 70:         boolean isParallel = activeSettings != null && activeSettings.isParallelEligible() &&
items.size() > 1;
 71:         int threadCount = isParallel ? Math.min(items.size(), Math.max(2,
Runtime.getRuntime().availableProcessors())) : 1;
 94:         if (item.getSvgContent() == null || !item.isProcessed() ||
 95:                 (activeSettings != null && !activeSettings.equals(item.getSettings()))) {
 97:             item.setSettings(new ConversionSettings(activeSettings));
 99:             VectorizationService.processImageItem(item);
100:         }
104:         File destFile = new File(outputDirectory, baseName + ".svg");
108:         while (destFile.exists()) { destFile = new File(outputDirectory, baseName + "_" +
counter + ".svg"); counter++; }
114:         try (FileWriter writer = new FileWriter(destFile, StandardCharsets.UTF_8)) {
115:             writer.write(item.getSvgContent());
116:         }
150:         exportHistory.add(new ExportRecord(outputDirectory, new ArrayList<>(savedFiles)));
159:     }
```

- **บรรทัด 70–71:** ตรวจสอบเงื่อนไข **Task 9** เพื่อกำหนดจำนวนเธรดของ `exportExecutor`
- **บรรทัด 94–100:** หากภาพใดยังไม่ได้แปลง หรือมีการตั้งค่าไม่ตรงกับการตั้งค่าปัจจุบัน จะสั่งแปลงใหม่ให้เป็นปัจจุบันก่อนบันทึกไฟล์
- **บรรทัด 104–116:** ตั้งชื่อไฟล์เป็น `ชื่อรูปเดิม.svg` (หากชื่อซ้ำจะเติม `_1.svg`) แล้วเขียนโค้ด SVG ลงดิสก์ด้วย UTF-8
- **บรรทัด 150:** บันทึกประวัติลงใน `exportHistory` เพื่อให้ผู้ใช้กดปุ่ม `Export History` ดูย้อนหลังและกดเปิดโฟลเดอร์ได้

บทที่ 6: ส่วนควบคุมหน้าจอ (Controllers)

ประกอบด้วย DropZoneController (หน้าแรก) และ MainController (หน้าแก้ไขหลัก)

ไฟล์ที่ 16: DropZoneController.java

`src/main/java/se233/vertor_magic/controller/DropZoneController.java`

> **บทบาทหน้าที่:** ควบคุมหน้าจอแรก (`drop-zone-view.fxml`) รับไฟล์จากการลากวาง (Drag & Drop) หรือปุ่มเลือกไฟล์ แล้วโหลดหน้าจอหลัก (`main-view.fxml`) พร้อมส่งรายการไฟล์ไปให้ `MainController`

```
25: public class DropZoneController {
27:     @FXML private VBox dropZoneContainer;
28:     @FXML private VBox dropCard;
29:     @FXML private Button btnBrowseFiles;
30:     @FXML private Button btnBrowseZip;
33:     @FXML public void onDropZoneOver(DragEvent event) { ... }
44:     @FXML public void onDropZoneExited(DragEvent event) { ... }
50:     @FXML public void onDropZoneDropped(DragEvent event) {
53:         if (db.hasFiles()) {
54:             List<File> files = db.getFiles();
55:             transitionToMainEditor(files);
57:         }
61:     }
64:     @FXML public void onBrowseFilesClicked(ActionEvent event) { ... }
77:     @FXML public void onBrowseZipClicked(ActionEvent event) { ... }
89:     private void transitionToMainEditor(List<File> files) {
91:         FXMLLoader loader = new FXMLLoader(Launcher.class.getResource("main-view.fxml"));
92:         Parent editorRoot = loader.load();
93:         MainController mainController = loader.getController();
95:         boolean loadedSuccessfully = mainController.importFiles(files);
96:         if (loadedSuccessfully) {
97:             dropZoneContainer.getScene().setRoot(editorRoot);
98:         }
106:     }
107: }
```

- **บรรทัด 33–61:** จัดการอีเวนต์ Drag & Drop เมื่อลากไฟล์เข้ากรอบจะเปลี่ยนสไตล์เป็น `drop-card-active` และเมื่อวางไฟล์จะส่งลิสต์ไฟล์เข้า `transitionToMainEditor(files)`
- **บรรทัด 64–87:** เปิดหน้าต่าง `FileChooser` กรองเฉพาะไฟล์รูปภาพ (`.png, .jpg, .jpeg`) หรือไฟล์ `.zip`
- **บรรทัด 89–106 (`transitionToMainEditor`):** โหลด `main-view.fxml` ดึง `MainController` ออกมาแล้วเรียก `mainController.importFiles(files)` หากโหลดไฟล์รูปสำเร็จ (`true`) จะสั่งสลับหน้าจอด้วย `setRoot(editorRoot)` ทันที

ไฟล์ที่ 17: MainController.java

`src/main/java/se233/vertor_magic/controller/MainController.java`

> **บทบาทหน้าที่:** คลาสควบคุมหลักของหน้าจอแก้ไขเวกเตอร์ (`main-view.fxml`) ทำหน้าที่เชื่อมโยงปุ่มและการตั้งค่าทั้งหมดเข้ากับเซอร์วิสเบื้องหลัง

สรุปการทำงานของทุกเมธอดใน MainController.java

- **`initialize()` (บรรทัด 87–100):** ตั้งค่า `Spinner` จำนวนสี (2–5, ค่าเริ่มต้น 2) พร้อมผูก Listener ให้เรียก `onSettingChanged()` เมื่อค่าเปลี่ยน และอัปเดตป้ายสถานะ Task 9
- **`onImportMoreClicked()` & `importFiles()` (บรรทัด 106–184):** รับลิสต์ไฟล์เข้ามา ส่งไฟล์ ZIP ให้ `ZipService.extractImages()` แปลงไฟล์รูปเป็นอ็อบเจกต์ `ImageItem` เก็บลง `imageItems` แล้วเรียก `startBatchPreprocessing()`
- **`startBatchPreprocessing()` (บรรทัด 190–210):** เรียก `batchProcessingService.preprocessBatch(...)` พร้อมส่ง Lambda Callback ไปอัปเดต `progressBar`, `lblStatus` และวาดรูปที่แปลงเสร็จขึ้นหน้าจอทันที
- **`onZoomInClicked()`, `onZoomOutClicked()`, `onZoomFitClicked()`, `applyZoom()` (บรรทัด 217–252):** ปรับค่าตัวคูณซูม `currentZoomFactor` (0.2x ถึง 5.0x) แล้วขยายทั้งภาพต้นฉบับ (`originalImageView`) และภาพเวกเตอร์ (`vectorGroup`) ให้มีขนาดตรงกัน
- **`onBackClicked()`, `onNextClicked()`, `updateNavigationButtons()` (บรรทัด 259–280):** เลื่อนลำดับรูปภาพ `currentIndex` เปิด/ปิดปุ่ม Back และ Next เมื่ออยู่รูปแรกหรือรูปสุดท้าย และแสดงเลขลำดับรูป (เช่น `1 / 5`)
- **`onDetailChanged()`, `onColorModeChanged()`, `onRemoveBackgroundChanged()`, `onSettingChanged()` (บรรทัด 287–313):** อ่านค่าจากปุ่มบนหน้าจอด้วย `buildCurrentSettingsFromUI()` บันทึกลง `current.setSettings(settings)` อัปเดตป้าย Task 9 และสั่ง `triggerInstantPreview(current)`
- **`triggerInstantPreview(ImageItem item)` (บรรทัด 331–362):** สร้างเธรดพื้นหลัง `Task<Void>` เรียก `VectorizationService.processImageItem(item)` เพื่อแปลงภาพปัจจุบันใหม่และอัปเดตหน้าจอทันทีโดยที่ UI ไม่ค้าง
- **`onExportAllClicked()` (บรรทัด 369–396):** เปิด `DirectoryChooser` ให้เลือกโฟลเดอร์ปลายทาง แล้วสั่ง `fileExportService.exportAll(...)`
- **`onResetSettingsClicked()` & `onHistoryClicked()` (บรรทัด 403–460):** รีเซ็ตการตั้งค่ากลับเป็นค่าเริ่มต้น (Medium + Custom 2 สี) และแสดงหน้าต่างประวัติการส่งออกพร้อมปุ่มกดเปิดโฟลเดอร์ล่าสุด
- **`updateEditorUI()` & `displayCurrentImage()` (บรรทัด 466–535):** ซิงค์ค่าจาก `ImageItem` ปัจจุบันขึ้นแสดงบนปุ่มควบคุม ปรับเพดานสูงสุดของ Spinner ตามจำนวนสีที่ตรวจพบในภาพ และนำโค้ด SVG ส่งให้ `SvgViewRenderer.renderSvgToGroup()` วาดขึ้นหน้าจอ

บทที่ 7: ไฟล์ออกแบบหน้าจอและสไตล์ (FXML Views & CSS)

ประกอบด้วย drop-zone-view.fxml, main-view.fxml และ styles.css

ไฟล์ที่ 18: drop-zone-view.fxml

`src/main/resources/se233/vertor_magic/drop-zone-view.fxml`

> **บทบาทหน้าที่:** ไฟล์โครงสร้าง UI หน้าแรกสำหรับรับไฟล์ (ผูกกับ `DropZoneController`) แยกออกมาเป็นไฟล์เดี่ยวเพื่อให้เปิดดูและแก้ไขใน Scene Builder ได้โดยตรง

- **บรรทัด 11:** โหนดราก `<VBox fx:id="dropZoneContainer">` ขนาด `1160 x 780` เชื่อมกับ `styles.css` และ `DropZoneController`
- **บรรทัด 13:** กล่องการ์ดเส้นประ `<VBox fx:id="dropCard">` ผูกอีเวนต์ `onDragOver="#onDropZoneOver"`, `onDragExited="#onDropZoneExited"`, และ `onDragDropped="#onDropZoneDropped"`
- **บรรทัด 22–23:** ปุ่ม `btnBrowseFiles` (เลือกไฟล์รูปภาพ) และ `btnBrowseZip` (เลือกไฟล์ ZIP)

ไฟล์ที่ 19: main-view.fxml

`src/main/resources/se233/vertor_magic/main-view.fxml`

> **บทบาทหน้าที่:** ไฟล์โครงสร้าง UI หน้าหลักของโปรแกรม (ผูกกับ `MainController`) ใช้ `BorderPane` แบ่งหน้าจอออกเป็น 4 ส่วนตามสเปกของโจทย์

- **ส่วนบน `<top>` (บรรทัด 25–33):** แถบเครื่องมือด้านบน มีเฉพาะปุ่ม `Zoom In`, `Zoom Out`, และ `Zoom to Fit` เพียง 3 ปุ่มตามข้อกำหนด **Task 2.B**
- **ส่วนกลาง `<center>` (บรรทัด 36–88):** `SplitPane` แบ่งหน้าจอเปรียบเทียบซ้าย-ขวา (**Task 2.A**):
  - ฝั่งซ้าย: `ScrollPane` บรรจุ `ImageView` (`originalImageView`) แสดงภาพต้นฉบับ
  - ฝั่งขวา: `ScrollPane` บรรจุ `Group` (`vectorGroup`) แสดงภาพเวกเตอร์ SVG
- **ส่วนขวา `<right>` (บรรทัด 91–172):** แผงตั้งค่า (**Task 2.C**) ประกอบด้วย:
  - **Image Navigation (Task 3):** ปุ่ม `Back`, ป้ายนับรูป `lblImageCounter`, ปุ่ม `Next`
  - **Detail Level (Task 4):** ปุ่มวิทยุ `Low`, `Medium`, `High` ในกลุ่ม `tgDetail`
  - **Colors (Task 5):** ปุ่มวิทยุ `Unlimited`, `Custom` และ `Spinner` (`spnColorCount`)
  - **Background (Task 6):** ช่องติ๊กถูก `Remove Background`
  - **Extra Tools & Batch Export (Task 8 & 12):** ปุ่ม `Reset Settings`, `Export History`, `Export All (SVG)`, และ `Import More Images`
- **ส่วนล่าง `<bottom>` (บรรทัด 175–183):** แถบแสดงสถานะการทำงาน (`lblStatus`), ป้ายแสดงโหมด Parallel/Sequential (`lblParallelStatus`), และหลอดความคืบหน้า (`progressBar`)

ไฟล์ที่ 20: styles.css

`src/main/resources/se233/vertor_magic/styles.css`

> **บทบาทหน้าที่:** กำหนดสีพื้นหลัง เส้นขอบ และขนาดตัวอักษรแบบเรียบง่ายให้กับทั้ง 2 หน้าจอ FXML

- `.root` (บรรทัด 1–4): กำหนดฟอนต์ `Arial, sans-serif` และพื้นหลังสีเทาอ่อน `#f4f4f4`
- `.drop-card` & `.drop-card-active` (บรรทัด 11–22): กรอบเส้นประสีเทาสำหรับวางไฟล์ และเปลี่ยนเป็นเส้นสีน้ำเงิน `#0066cc` เมื่อลากไฟล์มาวาง
- `.top-toolbar`, `.right-panel`, `.status-bar` (บรรทัด 44–100): แบ่งโซนพื้นที่การควบคุมด้วยเส้นขอบสีเทา `#cccccc`
- `.parallel-badge` & `.sequential-badge` (บรรทัด 104–117): แสดงสีพื้นหลังสีเขียวอ่อนเมื่อทำงานแบบ Parallel และสีเหลืองอ่อนเมื่อทำงานแบบ Sequential

ตารางสรุปการรับ-ส่งข้อมูลระหว่างไฟล์ทั้งระบบ

ลำดับ	ไฟล์ต้นทาง	ไฟล์ปลายทาง	ข้อมูลที่ส่งหากัน	ผลลัพธ์ที่ได้
1	`Launcher.java`	`drop-zone-view.fxml`	`Stage` & FXML Resource	เปิดหน้าต่างแรกสำหรับลากวางไฟล์
2	`DropZoneController`	`MainController`	`List<File>` ผ่าน `importFiles()`	โหลดหน้าหลักและสลับ Scene เข้าสู่ Editor
3	`MainController`	`ZipService`	`File` (.zip)	แตกไฟล์รูป `.jpg/.png` ออกมาเป็น `List<File>`
4	`MainController`	`ImageItem`	`File` รูปภาพ	อ่าน `BufferedImage`, `fxImage` และผูก `ConversionSettings`
5	`MainController`	`BatchProcessingService`	`List<ImageItem>` + Callbacks	เช็ก `isParallelEligible()` แล้วรันเธรดแปลงภาพล่วงหน้า
6	`VectorizationService`	`ImagePreconditioner`	`BufferedImage` + `tolerance`	ภาพที่กรอง Noise ออกและปรับขอบคมชัดแล้ว
7	`VectorizationService`	`ColorDetector` & `PaletteQuantizer`	`BufferedImage` + จำนวนสี	พาเลตต์สีในรูปแบบ `byte[][]` และสีพื้นหลังของภาพ
8	`VectorizationService`	`ImageTracer`	`ImageData`, `options`, `palette`	คำนวณเส้นโค้งและคืนค่าโค้ด SVG XML (พร้อมตัดพื้นหลังถ้าเลือก)
9	`MainController`	`SvgViewRenderer`	`String svgContent`	แปลงแท็ก `<path>` เป็น `Group` ของ `SVGPath` แสดงบนจอ
10	`MainController`	`FileExportService`	`List<ImageItem>` + โฟลเดอร์ปลายทาง	บันทึกไฟล์ `.svg` ลงดิสก์และเก็บประวัติการส่งออก